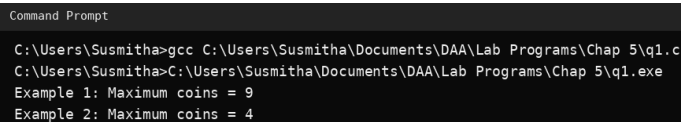


1. Maximum Coins from Piles

Program:

```
#include <stdio.h>
#include <stdlib.h>
int cmp(const void *a,const void *b){return *(int*)a-*(int*)b;}
int maximumCoins(int p[],int n){
    qsort(p,n,sizeof(int),cmp);
    int ans=0;
    for(int i=n-2;i>=n/3;i-=2) ans+=p[i];
    return ans;
}
int main(){
    int p1[]={2,4,1,2,7,8},p2[]={2,4,5};
    printf("Example 1: Maximum coins = %d\n",maximumCoins(p1,6));
    printf("Example 2: Maximum coins = %d\n",maximumCoins(p2,3));
    return 0;
}
```

Output:



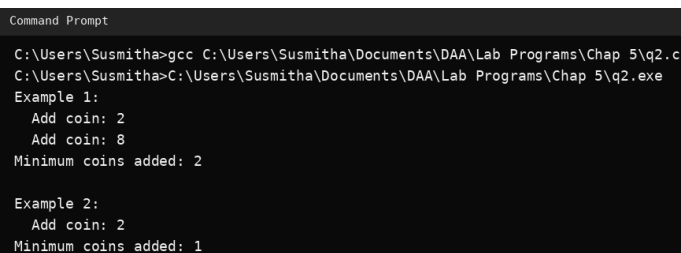
```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q1.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q1.exe
Example 1: Maximum coins = 9
Example 2: Maximum coins = 4
```

2. Minimum Coins to Reach Every Value

Program:

```
#include <stdio.h>
#include <stdlib.h>
int cmp(const void*a,const void*b){return *(int*)a-*(int*)b;}
int minAdded(int c[],int n,int target){
    qsort(c,n,sizeof(int),cmp);
    long long reach=1; int i=0,added=0;
    while(reach<=target){
        if(i<n && c[i]<=reach) reach+=c[i++];
        else {printf(" Add coin: %lld\n",reach); reach*=2; added++;}
    }
    return added;
}
int main(){
    int c1[]={1,4,10},c2[]={1,4,10,5,7,19};
    printf("Example 1:\n");
    printf("Minimum coins added: %d\n",minAdded(c1,3,19));
    printf("\nExample 2:\n");
    printf("Minimum coins added: %d\n",minAdded(c2,6,19));
    return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q2.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q2.exe
Example 1:
  Add coin: 2
  Add coin: 8
Minimum coins added: 2

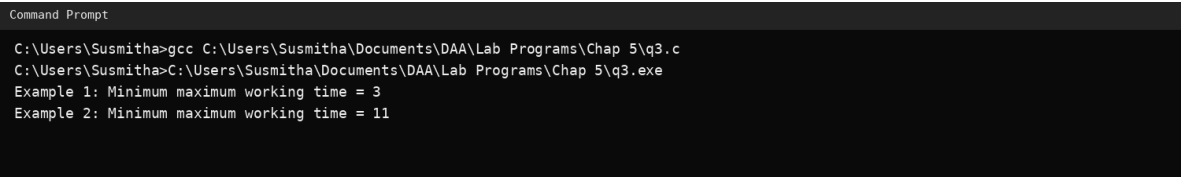
Example 2:
  Add coin: 2
Minimum coins added: 1
```

3. Minimum Maximum Working Time

Program:

```
#include <stdio.h>
#include <limits.h>
int best,k,n,a[20],load[20];
void dfs(int idx){
    if(idx==n){
        int mx=0;
        for(int i=0;i<k;i++) if(load[i]>mx) mx=load[i];
        if(mx<best) best=mx;
        return;
    }
    for(int i=0;i<k;i++){
        load[i]+=a[idx];
        if(load[i]<best) dfs(idx+1);
        load[i]-=a[idx];
    }
}
int solve(int jobs[],int count,int workers){
    n=count;k=workers;best=INT_MAX;
    for(int i=0;i<n;i++)a[i]=jobs[i];
    for(int i=0;i<k;i++)load[i]=0;
    dfs(0); return best;
}
int main(){
    int a1[]={3,2,3},a2[]={1,2,4,7,8};
    printf("Example 1: Minimum maximum working time = %d\n",solve(a1,3,3));
    printf("Example 2: Minimum maximum working time = %d\n",solve(a2,5,2));
    return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q3.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q3.exe
Example 1: Minimum maximum working time = 3
Example 2: Minimum maximum working time = 11
```

4. Maximum Profit in Job Scheduling

Program:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct{int s,e,p;} Job;
int cmp(const void*a,const void*b){return ((Job*)a)->e-((Job*)b)->e;}
int latest(Job j[],int i){
    for(int k=i-1;k>=0;k--) if(j[k].e<=j[i].s) return k;
    return -1;
}
int solve(Job j[],int n){
    qsort(j,n,sizeof(Job),cmp);
    int dp[50]={0};
    for(int i=1;i<=n;i++){
        int incl=j[i-1].p, k=latest(j,i-1);
        if(k>=0) incl+=dp[k+1];
        dp[i]=incl>dp[i-1]?incl:dp[i-1];
    }
    return dp[n];
}
```

```

}
int main(){
    Job a[]={ {1,3,50}, {2,4,10}, {3,5,40}, {3,6,70} };
    Job b[]={ {1,3,20}, {2,5,20}, {3,10,100}, {4,6,70}, {6,9,60} };
    printf("Example 1: Maximum profit = %d\n",solve(a,4));
    printf("Example 2: Maximum profit = %d\n",solve(b,5));
    return 0;
}

```

Output:

```

Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q4.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q4.exe
Example 1: Maximum profit = 120
Example 2: Maximum profit = 150

```

5. Dijkstra's Algorithm using Adjacency Matrix

Program:

```

#include <stdio.h>
#define INF 999999
void dijkstra(int g[][5],int n,int src,int d[]){
    int vis[5]={0};
    for(int i=0;i<n;i++) d[i]=INF;
    d[src]=0;
    for(int c=0;c<n;c++){
        int u=-1;
        for(int i=0;i<n;i++) if(!vis[i]&&(u==-1||d[i]<d[u])) u=i;
        if(u==-1||d[u]==INF) break;
        vis[u]=1;
        for(int v=0;v<n;v++){
            if(g[u][v]<INF && d[u]+g[u][v]<d[v])
                d[v]=d[u]+g[u][v];
        }
    }
}
void print(int d[],int n){for(int i=0;i<n;i++)printf("%d%s",d[i],i==n-1?"\n":" ");}
int main(){
    int
    g1[5][5]={ {0,10,3,INF,INF}, {INF,0,1,2,INF}, {INF,4,0,8,2}, {INF,INF,INF,0,7}, {INF,INF,INF,9,0} };
    int g2[5][5]={ {0,5,INF,10,}, {INF,0,3,INF}, {INF,INF,0,1}, {INF,INF,INF,0}, {0,0,0,0,0} };
    int
    h2[5][5]={ {0,5,INF,10,INF}, {INF,0,3,INF,INF}, {INF,INF,0,1,INF}, {INF,INF,INF,0,INF}, {INF,INF,INF,INF,0} };
    int d[5];
    dijkstra(g1,5,0,d); printf("Example 1: "); print(d,5);
    dijkstra(h2,4,0,d); printf("Example 2: "); print(d,4);
    return 0;
}

```

Output:

```

Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q5.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q5.exe
Example 1: 0 7 3 9 5
Example 2: 0 5 8 9

```

6. Dijkstra's Algorithm using Edge List

Program:

```

#include <stdio.h>
#define INF 999999
typedef struct{int u,v,w;} Edge;
void add(Edge e[],int *m,int u,int v,int w){e[(*m)++] = (Edge){u,v,w};e[(*m)++] = (Edge){v,u,w};}
void dijkstra(Edge e[],int m,int n,int src,int d[]){
    int vis[20]={0};
    for(int i=0;i<n;i++)d[i]=INF;
    d[src]=0;
    for(int c=0;c<n;c++){
        int u=-1;
        for(int i=0;i<n;i++)if(!vis[i]&&(u==-1||d[i]<d[u]))u=i;
        if(u==-1||d[u]==INF)break;
        vis[u]=1;
        for(int i=0;i<m;i++)if(e[i].u==u&&d[u]+e[i].w<d[e[i].v])
            d[e[i].v]=d[u]+e[i].w;
    }
}
int main(){
    Edge a[20];int ma=0;
    int raw1[][3]={ {0,1,7},{0,2,9},{0,5,14},{1,2,10},{1,3,15},{2,3,11},{2,5,2},{3,4,6},{4,5,9}};
    for(int i=0;i<9;i++)add(a,&ma,raw1[i][0],raw1[i][1],raw1[i][2]);
    Edge b[20];int mb=0;
    int raw2[][3]={ {0,1,10},{0,4,3},{1,2,2},{1,4,4},{2,3,9},{3,2,7},{4,1,1},{4,2,8},{4,3,2}};
    for(int i=0;i<9;i++)add(b,&mb,raw2[i][0],raw2[i][1],raw2[i][2]);
    int d[6];
    dijkstra(a,ma,6,0,d);printf("Example 1: Shortest path from 0 to 4 = %d\n",d[4]);
    dijkstra(b,mb,5,0,d);printf("Example 2: Shortest path from 0 to 3 = %d\n",d[3]);
    return 0;
}

```

Output:

```

Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q6.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q6.exe
Example 1: Shortest path from 0 to 4 = 20
Example 2: Shortest path from 0 to 3 = 5

```

7. Huffman Tree and Codes

Program:

```

#include <stdio.h>
typedef struct Node{char c;int f;struct Node*l,*r;}Node;
Node pool[30];int pc;
Node* make(char c,int f,Node*l,Node*r){Node*x=&pool[pc++];x->c=c;x->f=f;x->l=l;x->r=r;return x;}
void huff(char ch[],int f[],int n){
    Node*q[30];int m=0;
    for(int i=0;i<n;i++)q[m++]=make(ch[i],f[i],NULL,NULL);
    while(m>1){
        int x=-1,y=-1;
        for(int i=0;i<m;i++)
            if(x==-1||q[i]->f<q[x]->f)x=i;
        for(int i=0;i<m;i++)
            if(i!=x&&(y==-1||q[i]->f<q[y]->f))y=i;
        Node*a=q[x],*b=q[y],*z=make('\0',a->f+b->f,a,b);
        if(x>y){int t=x;x=y;y=t;}
        q[x]=z;q[y]=q[m-1];m--;
    }
    char code[50];
}

```

```

void dfs(Node*t,int d){
    if(!t->l&&!t->r){code[d]='\0';printf("%c: %s\n",t->c,code);return;}
    code[d]='0';dfs(t->l,d+1);
    code[d]='1';dfs(t->r,d+1);
}
dfs(q[0],0);
}
int main(){
    char c1[]={'a','b','c','d'};int f1[]={5,9,12,13};
    char c2[]={'f','e','d','c','b','a'};int f2[]={5,9,12,13,16,45};
    printf("Example 1 - Huffman Codes:\n");pc=0;huff(c1,f1,4);
    printf("Example 2 - Huffman Codes:\n");pc=0;huff(c2,f2,6);
    return 0;
}

```

Output:

```

Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q7.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q7.exe
Example 1 - Huffman Codes:
a: 00
b: 01
c: 10
d: 11
Example 2 - Huffman Codes:
a: 0
d: 100
c: 101
f: 1100
e: 1101
b: 111

```

8. Huffman Decoding

Program:

```

#include <stdio.h>
#include <string.h>
void decode(const char*s,const char*codes[],const char chars[],int n){
    int i=0;
    while(s[i]){
        int found=0;
        for(int j=0;j<n;j++){
            int l=(int)strlen(codes[j]);
            if(strncmp(s+i,codes[j],l)==0){printf("%c",chars[j]);i+=l;found=1;break;}
        }
        if(!found){printf("[invalid]");break;}
    }
    printf("\n");
}
int main(){
    /* These are the Huffman codes generated by Experiment 7 for the same frequencies. */
    const char*c1[]={"00","01","10","11"};const char ch1[]="abcd";
    const char*c2[]={"1100","1101","100","101","111","0"};const char ch2[]="fedcba";

    printf("Example 1: Encoded string for \"abacd\" = 0001001011\n");
    printf("Decoded message: ");decode("0001001011",c1,ch1,4);

    printf("Example 2: Encoded string for \"fcbade\" = 110010111101001101\n");
    printf("Decoded message: ");decode("110010111101001101",c2,ch2,6);
    return 0;
}

```

Output:

```

Command Prompt

C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q8.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q8.exe
Example 1: Encoded string for "abacd" = 0001001011
Decoded message: abacd
Example 2: Encoded string for "fcbade" = 110010111101001101
Decoded message: fcbade

```

9. Maximum Weight using Greedy Approach

Program:

```

#include <stdio.h>
#include <stdlib.h>
int cmp(const void*a,const void*b){return *(int*)b-*(int*)a;}
int maxWeight(int w[],int n,int cap){
    qsort(w,n,sizeof(int),cmp);
    int sum=0;
    for(int i=0;i<n;i++){
        if(sum+w[i]>cap) continue;
        sum+=w[i];
    }
    return sum;
}
int main(){
    int a[]={10,20,30,40,50},b[]={5,10,15,20,25,30};
    printf("Example 1: Maximum loaded weight = %d\n",maxWeight(a,5,60));
    printf("Example 2: Maximum loaded weight = %d\n",maxWeight(b,6,50));
    return 0;
}

```

Output:

```

Command Prompt

C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q9.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q9.exe
Example 1: Maximum loaded weight = 60
Example 2: Maximum loaded weight = 50

```

10. Minimum Number of Containers

Program:

```

#include <stdio.h>
int containers(int w[],int n,int cap){
    int used=0,cur=0;
    for(int i=0;i<n;i++){
        if(cur+w[i]>cap){used++;cur=0;}
        cur+=w[i];
    }
    if(cur)used++;
    return used;
}
int main(){
    int a[]={5,10,15,20,25,30,35},b[]={10,20,30,40,50,60,70,80};
    printf("Example 1: Minimum containers required = %d\n",containers(a,7,50));
    printf("Example 2: Minimum containers required = %d\n",containers(b,8,100));
    return 0;
}

```

Output:

```

Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q10.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q10.exe
Example 1: Minimum containers required = 4
Example 2: Minimum containers required = 5

```

11. Kruskal's Algorithm for MST

Program:

```

#include <stdio.h>
typedef struct {int u,v,w;} Edge;
int p[20];
int find(int x){return p[x]==x?(p[x]=find(p[x]));}
int uni(int a,int b){a=find(a);b=find(b);if(a==b)return 0;p[a]=b;return 1;}
void sort(Edge e[],int m){for(int i=0;i<m;i++)for(int j=i+1;j<m;j++)if(e[j].w<e[i].w){Edge
t=e[i];e[i]=e[j];e[j]=t;}}
void kruskal(Edge e[],int m,int n){
    sort(e,m);for(int i=0;i<n;i++)p[i]=i;int total=0,count=0;
    for(int i=0;i<m&&count<n-1;i++)if(uni(e[i].u,e[i].v)){
        printf("%d,%d,%d) ",e[i].u,e[i].v,e[i].w);total+=e[i].w;count++;
    }
    printf("\nTotal weight of MST: %d\n",total);
}
int main(){
    Edge a[]={ {0,1,10},{0,2,6},{0,3,5},{1,3,15},{2,3,4}};
    Edge b[]={ {0,1,2},{0,3,6},{1,2,3},{1,3,8},{1,4,5},{2,4,7},{3,4,9}};
    printf("Example 1 - Edges in MST: ");kruskal(a,5,4);
    printf("Example 2 - Edges in MST: ");kruskal(b,7,5);
    return 0;
}

```

Output:

```

Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q11.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q11.exe
Example 1 - Edges in MST: (2,3,4) (0,3,5) (0,1,10)
Total weight of MST: 19
Example 2 - Edges in MST: (0,1,2) (1,2,3) (1,4,5) (0,3,6)
Total weight of MST: 16

```

12. Verify Uniqueness of an MST

Program:

```

#include <stdio.h>
typedef struct {int u,v,w;} Edge;
int find(int p[],int x){return p[x]==x?(p[x]=find(p,p[x]));}
int unite(int p[],int a,int b){a=find(p,a);b=find(p,b);if(a==b)return 0;p[a]=b;return 1;}
int sameSet(int a[],int n,int b[],int m){
    if(n!=m)return 0;
    for(int i=0;i<n;i++){int found=0;for(int j=0;j<m;j++)if(a[i]==b[j])found=1;if(!found)return 0;}
    return 1;
}
void verify(Edge e[],int m,int n,int given[],int gc){
    int gw=0;for(int i=0;i<gc;i++)gw+=e[given[i]].w;
    int alt[10],foundAlt=0;
    int need=n-1;
    if(gc!=need){printf("Invalid given MST size.\n");return;}
    int idx[10];
    if(n==4){
        for(int a=0;a<m;a++)for(int b=a+1;b<m;b++)for(int c=b+1;c<m;c++){
            int ids[3]={a,b,c},p[10];for(int i=0;i<n;i++)p[i]=i;
            int ok=1,w=0;for(int i=0;i<3;i++){if(!unite(p,e[ids[i]].u,e[ids[i]].v))ok=0;w+=e[ids[i]].w;}
        }
    }
}

```

```

        if(ok&&w==gw&&!sameSet(ids,3,given,gc)){for(int i=0;i<3;i++)alt[i]=ids[i];foundAlt=1;}
    }
} else {
    for(int a=0;a<m;a++)for(int b=a+1;b<m;b++)for(int c=b+1;c<m;c++)for(int d=c+1;d<m;d++){
        int ids[4]={a,b,c,d},p[10];for(int i=0;i<n;i++)p[i]=i;
        int ok=1,w=0;for(int i=0;i<4;i++){if(!unite(p,e[ids[i]].u,e[ids[i]].v))ok=0;w+=e[ids[i]].w;}
        if(ok&&w==gw&&!sameSet(ids,4,given,gc)){for(int i=0;i<4;i++)alt[i]=ids[i];foundAlt=1;}
    }
}
printf("Is the given MST unique? %s\n",foundAlt?"False":"True");
if(foundAlt){
    printf("Another possible MST: [");
    for(int i=0;i<need;i++)printf("(%d,%d,%d)%s",e[alt[i]].u,e[alt[i]].v,e[alt[i]].w,i==need-1?"":", ");
    printf("]\nTotal weight of MST: %d\n",gw);
}
}
int main(){
    Edge e1[]={{0,1,10},{0,2,6},{0,3,5},{1,3,15},{2,3,4}};
    int g1[]={4,2,0};
    printf("Example 1:\n");verify(e1,5,4,g1,3);

    Edge e2[]={{0,1,1},{0,2,1},{1,3,2},{2,3,2},{3,4,3},{4,2,3}};
    int g2[]={0,1,2,4};
    printf("\nExample 2:\n");verify(e2,6,5,g2,4);
    return 0;
}

```

Output:

```

Command Prompt

C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q12.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 5\q12.exe
Example 1:
Is the given MST unique? True

Example 2:
Is the given MST unique? False
Another possible MST: [(0,1,1), (0,2,1), (2,3,2), (4,2,3)]
Total weight of MST: 7

```